CS 598 PSL Practical Statistical Learning — Fall 2026

Coding Assignment 3

Author: David Gray

Data loading

In [1]:
```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.metrics import mean_squared_error
from sklearn.tree import DecisionTreeRegressor, plot_tree
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressc
from sklearn.linear_model import LinearRegression
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.pipeline import Pipeline
import matplotlib.pyplot as plt
```

In [2]:
```
df = pd.read_csv("CollegeScores4yr.csv")
df.dropna(subset=['CompRate'], inplace=True)
df.drop(columns=['Accred', 'State', 'MainDegree'], inplace=True)
cols_with_na = ['AdmitRate', 'MidACT', 'AvgSAT']
na_flags = df[cols_with_na].isna().astype(int).add_suffix('_na')
df = pd.concat([df, na_flags], axis=1)
df.head()
```

Out[2]:

	Name	ID	Main	HighDegree	Control	Region	Locale	Latitude	Lo
0	Alabama A & M University	100654	1	4	Public	Southeast	City	34.783368	-86
1	University of Alabama at Birmingham	100663	1	4	Public	Southeast	City	33.505697	-86
2	Amridge University	100690	1	4	Private	Southeast	City	32.362609	-8
3	University of Alabama in Huntsville	100706	1	4	Public	Southeast	City	34.724557	-86
4	Alabama State University	100724	1	4	Public	Southeast	City	32.364317	-86

5 rows × 35 columns

Part I: Tree-based methods

In [3]:

```
# a
train_df, test_df = train_test_split(df, test_size=0.3, random_state=598)
meta_train = train_df[['ID', 'Name']]
meta_test = test_df[['ID', 'Name']]

train_df = train_df.drop(columns=['ID', 'Name']).copy()
test_df = test_df.drop(columns=['ID', 'Name']).copy()

# Fill numerical n/a with median
medians = train_df.median(numeric_only=True)
train_df.fillna(medians, inplace=True)
test_df.fillna(medians, inplace=True)

# Fill categorical n/a with mode
modes = train_df.select_dtypes(include=['object', 'str']).mode().iloc[0]
train_df.fillna(modes, inplace=True)
test_df.fillna(modes, inplace=True)

# Map any unseen categories in test to training mode
for col in modes.index:
    train_levels = set(train_df[col].unique())
    mode_val = modes[col]
    test_df[col] = test_df[col].apply(lambda x: x if x in train_levels else

train_df
```

Out[3]:

	Main	HighDegree	Control	Region	Locale	Latitude	Longitude	AdmitR
1694	1	4	Private	West	City	35.347785	-97.578297	77
529	1	3	Public	Southeast	Rural	31.178616	-92.414409	57
719	1	4	Public	Midwest	City	44.972851	-93.235464	74
401	1	4	Private	Midwest	Town	39.823431	-84.915037	73
1094	1	4	Public	Midwest	Town	46.882917	-102.799679	72
...	...	...	...	...	...	...	...	
1162	1	4	Private	Midwest	City	39.119091	-84.497652	77
306	1	4	Private	Southeast	Town	34.593414	-83.357617	68
422	1	4	Private	Midwest	City	39.813233	-86.202643	93
178	1	3	Private	West	City	38.944824	-104.803251	77
1963	1	4	Public	Midwest	Suburb	41.584324	-87.474236	72

1278 rows × 33 columns

In [4]:
```
models = {}
```

In [5]:
```
# a.i
global_mean = train_df['CompRate'].mean()
test_pred_i = pd.Series(global_mean, index=test_df.index)
mse_i = mean_squared_error(test_df['CompRate'], test_pred_i)
models['global_mean'] = {'mse': mse_i}

# a.ii
control_means = train_df.groupby('Control')['CompRate'].mean()
test_pred_ii = test_df['Control'].map(control_means)
mse_ii = mean_squared_error(test_df['CompRate'], test_pred_ii)
models['control_mean'] = {'mse': mse_ii}

print(f"Baseline (i) Test MSE: {mse_i:.5f}")
print(f"Baseline (ii) Test MSE: {mse_ii:.5f}")
```

```
Baseline (i) Test MSE: 405.40954
Baseline (ii) Test MSE: 397.95367
```

In [6]:
```
# One hot encode categorical data
train_df_enc = pd.get_dummies(train_df, columns=modes.index, dtype=int)
test_df_enc = pd.get_dummies(test_df, columns=modes.index, dtype=int)

X_train = train_df_enc.drop(columns=['CompRate']).copy()
y_train = train_df_enc['CompRate'].copy()

X_test = test_df_enc.drop(columns=['CompRate']).copy()
y_test = test_df_enc['CompRate'].copy()

train_df_enc.head()
```

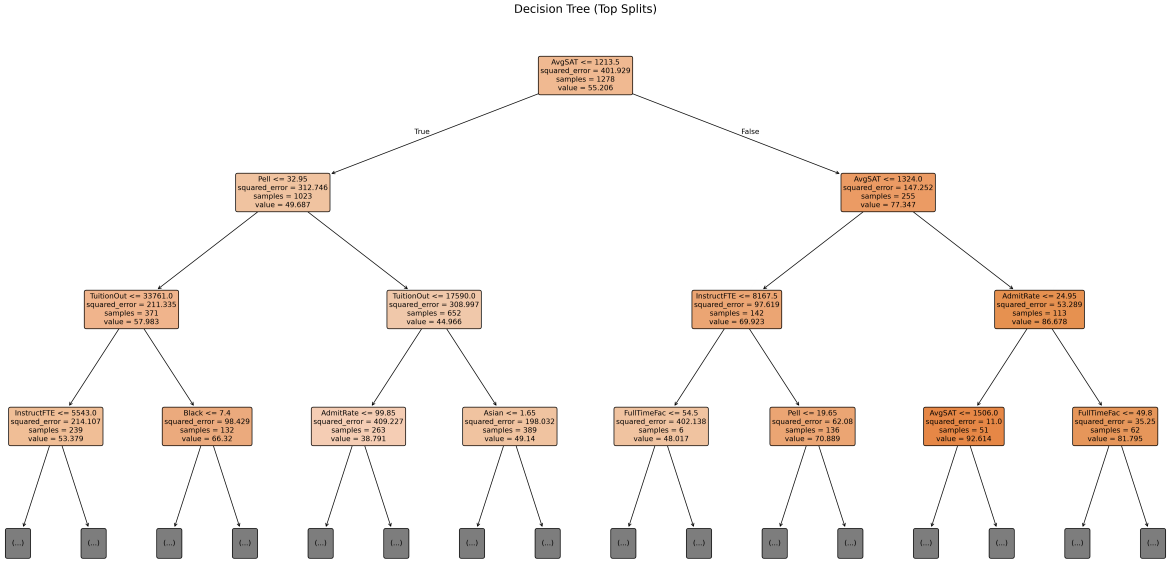
Out[6]:

	Main	HighDegree	Latitude	Longitude	AdmitRate	MidACT	AvgSAT	Onlin
1694	1	4	35.347785	-97.578297	77.65	24.0	1150.0	
529	1	3	31.178616	-92.414409	57.70	20.0	1019.0	
719	1	4	44.972851	-93.235464	74.90	30.0	1355.0	
401	1	4	39.823431	-84.915037	73.20	28.0	1261.0	
1094	1	4	46.882917	-102.799679	72.00	19.0	1018.0	

5 rows × 42 columns

In [7]:
```
# b
tree_b = DecisionTreeRegressor(random_state=598)
path_b = tree_b.cost_complexity_pruning_path(X_train, y_train)
tree_b.fit(X_train, y_train)
test_pred_b = tree_b.predict(X_test)
mse_b = mean_squared_error(y_test, test_pred_b)
models['tree'] = {'mse': mse_b}

plt.figure(figsize=(24, 12), dpi=300)  # Large size + high resolution
plot_tree(
    tree_b,
    feature_names=X_train.columns,
    max_depth=3,           # Shows only the top 3 levels so text isn't micro
    filled=True,           # Colors nodes by predicted value
    rounded=True,
    fontsize=10,
    precision=3
)
plt.title("Decision Tree (Top Splits)", fontsize=16)
plt.tight_layout()
plt.show()
```



Interpretation of Tree Model (b)

- **Split 1**: AvgSAT <= 1213.5
  - Schools with average SAT scores above 1213.5 split to the right and are predicted a completion rate of ~77%, while schools with lower SAT scores split left and are predicted a lower completion rate of ~50%.
  - This split reflects selectivity by measuring SAT scores.
- **Split 2**: Pell <= 32.95
  - The left child node of split 1 reflects student mix by using Pell Grant share of the students.
  - Schools with a lower share of Pell Grant students have a greater completion rate of ~58 in contrast with ~44 for schools serving higher need student populations.

In [8]:
```
# c
alphas_b = path_b.ccp_alphas
alphas_b = alphas_b[alphas_b >= 0][:-1]
param_grid = {'ccp_alpha': alphas_b}
grid_search = GridSearchCV(
    estimator=DecisionTreeRegressor(random_state=598),
    param_grid=param_grid,
    cv=5,
    n_jobs=-1
)
grid_search.fit(X_train, y_train)
test_pred_c = grid_search.predict(X_test)
mse_c = mean_squared_error(y_test, test_pred_c)
models['pruned_tree'] = {'mse': mse_c}

print(f"Unpruned Test MSE: {mse_b:.5f}")
print(f"Pruned Test MSE: {mse_c:.5f}")

train_public_mask = X_train['Control_Public'] == 1
X_train_public = X_train[train_public_mask]
y_train_public = y_train[train_public_mask]

tree_2c = DecisionTreeRegressor(random_state=598)
tree_2c.fit(X_train_public, y_train_public)
test_pred_2c = tree_2c.predict(X_test)
mse_2c = mean_squared_error(y_test, test_pred_2c)
models['public_tree'] = {'mse': mse_2c}

print(f"Public unpruned Test MSE: {mse_2c:.5f}")
```

```
Unpruned Test MSE: 320.42303
Pruned Test MSE: 210.12378
Public unpruned Test MSE: 436.90664
```

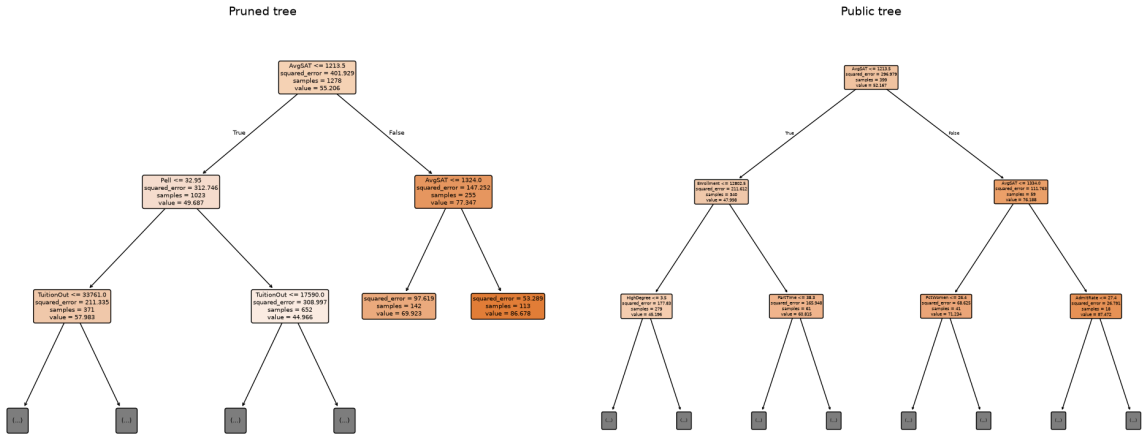
In [9]:
```
fig, axes = plt.subplots(nrows=1, ncols=2, figsize=(20, 8))

plot_tree(
    grid_search.best_estimator_,
```

```
    feature_names=X_train.columns,
    filled=True,
    rounded=True,
    max_depth=2,
    ax=axes[0]
)
axes[0].set_title("Pruned tree", fontsize=14)

plot_tree(
    tree_2c,
    feature_names=X_train.columns,
    filled=True,
    rounded=True,
    max_depth=2,
    ax=axes[1]
)
axes[1].set_title("Public tree", fontsize=14)

# 4. Adjust layout and display
plt.tight_layout()
plt.show()
```



Comparison of Pruned Tree vs. Public-Only Tree (c)

- **Pruning Effect**: Pruning the pooled decision tree using cost-complexity pruning reduced the test MSE from **320.42** (unpruned) to **210.12** (pruned), eliminating overfitted leaf nodes.
- **First Two Splits Comparison**:
  - **Root Split**: Both the pooled tree and the public-only tree split first on `AvgSAT <= 1213.5`, confirming that student academic selectivity is the most powerful initial differentiator of graduation rates across all institutions.
  - **Second Split**: In the pooled tree, the second split on the left branch is `Pell <= 32.95` (student financial/demographic mix) along with `TuitionOut`. In contrast, the public-only tree splits on `Enrollment <= 12802.5` on the left branch.

- **Comment on What Changed**: The pooled tree averages across Private and For-Profit institutions, where tuition and socioeconomic mix vary dramatically. Within the public sector, however, campus scale (`Enrollment`) strongly differentiates major state research flagships from smaller regional campuses in terms of institutional resources, student retention programs, and graduation outcomes.

In [10]:
```
# d

full_features = len(X_train.columns)
full_forest = RandomForestRegressor(max_features=full_features, random_state
full_forest.fit(X_train, y_train)

half_features = int(len(X_train.columns) / 2)
half_forest = RandomForestRegressor(max_features=half_features, random_state
half_forest.fit(X_train, y_train)
```

Out[10]:

▾ RandomForestRegressor ⓘ ?

▸ Parameters

▸ Fitted attributes

Random Forest Evaluation (d)

I evaluated random forests with two feature subsampling levels: `max_features = 41` (all features, equivalent to bagging) and `max_features = 20` (approximately half the features):

- **Full Features (`max_features = 41`)**: Test MSE = **154.62**
- **Half Features (`max_features = 20`)**: Test MSE = **153.39**

Restricting `max_features` to half of the predictors achieves a lower test MSE. By considering a random subset of candidate predictors at each split, the individual trees are decorrelated—preventing strong, dominant predictors (like `AvgSAT` and `TuitionOut`) from consistently monopolizing the top splits across all trees. This increases ensemble diversity and reduces overall prediction variance.

In [11]:
```
# e
rf_importances = pd.Series(half_forest.feature_importances_, index=X_train.c
display(rf_importances.head(10))
```

```
AvgSAT         0.197261
TuitionOut     0.136126
Pell           0.099053
Cost           0.042119
Black          0.041271
InstructFTE    0.040788
MidACT         0.037418
FacSalary      0.035938
AdmitRate      0.035037
TuitionFTE     0.030650
dtype: float64
```

Predictor Importance in Random Forest vs. Single Tree (e)

- **Most Important Predictors**: In the random forest (`max_features=20`), the most important predictors are **`AvgSAT`** (~0.20), **`TuitionOut`** (~0.14), **`Pell`** (~0.10), **`Cost`** (~0.04), **`Black`** (~0.04), **`InstructFTE`** (~0.04), **`MidACT`** (~0.04), and **`FacSalary`** (~0.04).
- **Predictor Important in Forest but Barely in Single Tree**: **`MidACT`** (as well as resource measures like **`Cost`** and **`InstructFTE`**).
- **Why this happens**: In a single tree, once `AvgSAT` is chosen at the root, collinear predictors like `MidACT` are masked and rarely selected; in a random forest, random feature subsampling forces trees to evaluate split candidates without `AvgSAT` present, allowing correlated predictors to be selected and demonstrate their predictive value.

In [12]:
```
# f
models['randforest_1'] = {'max_features': full_features}
models['randforest_2'] = {'max_features': half_features}

test_pred_full_forest = full_forest.predict(X_test)
mse_full_forest = mean_squared_error(y_test, test_pred_full_forest)
models['randforest_1']['mse'] = mse_full_forest

test_pred_half_forest = half_forest.predict(X_test)
mse_half_forest = mean_squared_error(y_test, test_pred_half_forest)
models['randforest_2']['mse'] = mse_half_forest

print(f"Random Forest 1 (max_features={full_features}) test MSE: {mse_full_f
print(f"Random Forest 2 (max_features={half_features}) test MSE: {mse_half_f
```

```
Random Forest 1 (max_features=41) test MSE: 154.61830
Random Forest 2 (max_features=20) test MSE: 153.38788
```

Both the random forests have significantly lower test MSE than the pruned tree's $MSE=210.12378$. There is roughly a difference of 50 in mean squared error!

In [13]:
```
for school_type in ['Public', 'Private', 'Profit']:
    col_name = 'Control_' + school_type
    header = col_name + '_mse'
```

```
    test_school_type_mask = X_test[col_name] == 1
    X_test_school = X_test[test_school_type_mask]
    y_test_school = y_test[test_school_type_mask]

    # Pruned tree MSE by control
    test_pred_school_pruned = grid_search.predict(X_test_school)
    school_mse_pruned = mean_squared_error(y_test_school, test_pred_school_p
    models['pruned_tree'][header] = school_mse_pruned

    # Random Forest 1 (full features)
    test_pred_school_full_forest = full_forest.predict(X_test_school)
    school_mse_full_forest = mean_squared_error(y_test_school, test_pred_sch
    models['randforest_1'][header] = school_mse_full_forest

    # Random Forest 2 (half features)
    test_pred_school_half_forest = half_forest.predict(X_test_school)
    school_mse_half_forest = mean_squared_error(y_test_school, test_pred_sch
    models['randforest_2'][header] = school_mse_half_forest

display(pd.DataFrame.from_dict(models, orient='index').loc[['pruned_tree', '
```

	mse	Control_Public_mse	Control_Private_mse	Control_Profit_mse
pruned_tree	210.123777	165.375597	202.346976	468.790360
randforest_1	154.618297	63.644859	180.231237	333.570041
randforest_2	153.387885	65.825654	176.444005	338.999525

Comparison by Control Level (f)

Comparing the pruned tree with the random forests across institutional sectors:

1. **Overall Error Reduction**: The random forest (~153.39) significantly outperforms the pruned tree (~210.12), reducing overall test MSE by nearly 57 points.
2. **Error by Sector**:
   - **Public Institutions**: Both models achieve their lowest MSE here, but the random forest shows a dramatic improvement over the pruned tree (MSE drops from **165.38** to **65.83**, a ~60% reduction in prediction error).
   - **Private Institutions**: Error drops moderately from **202.35** (pruned tree) to **176.44** (random forest).
   - **For-Profit Institutions**: Both models perform worst on for-profit schools, with high test MSEs (**468.79** for the tree and **339.00** for the forest).
3. **Observations**: Public schools follow more consistent enrollment, state funding, and admission patterns that the forest captures well. In contrast, for-profit institutions have much higher variance and unpredictability due to non-traditional student populations, differing vocational missions, and frequent missing test scores.

In [14]:
```
# g
param_grid = {
```

```
    'learning_rate': [0.3, 0.1, 0.05, 0.01],
    'n_estimators': [100, 200, 300, 500, 800, 1000]
}

gbm_grid = GridSearchCV(
    estimator=GradientBoostingRegressor(random_state=598),
    param_grid=param_grid,
    cv=5,
    n_jobs=-1
)
gbm_grid.fit(X_train, y_train)

print("Best Parameters:", gbm_grid.best_params_)
best_gbm = gbm_grid.best_estimator_

# Evaluate ONCE on the test set
y_test_gbm = best_gbm.predict(X_test)
test_mse = mean_squared_error(y_test, y_test_gbm)
print("Test MSE:", test_mse)
models['gbm'] = {'mse': test_mse}

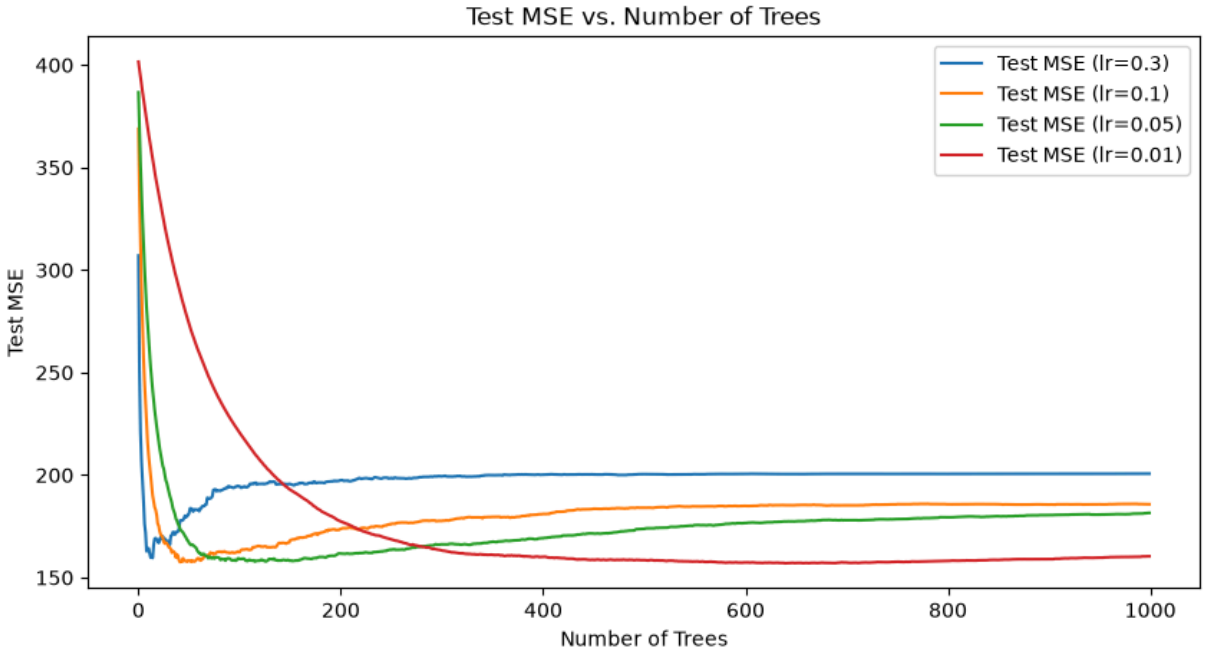
gbm_importances = pd.Series(best_gbm.feature_importances_, index=X_train.col
display(gbm_importances.head(10))
```

```
Best Parameters: {'learning_rate': 0.01, 'n_estimators': 1000}
Test MSE: 160.38425090594657
AvgSAT         0.273925
Pell           0.126342
TuitionOut     0.117347
InstructFTE    0.051838
AdmitRate      0.044284
Black          0.043475
FacSalary      0.042456
Cost           0.031895
Asian          0.028779
Enrollment     0.028366
dtype: float64
```

In [15]:
```
plt.figure(figsize=(10, 5))

for lr in param_grid['learning_rate']:
    m = GradientBoostingRegressor(n_estimators=1000,learning_rate=lr, random
    m.fit(X_train, y_train)
    test_errors = [mean_squared_error(y_test, y_pred) for y_pred in m.staged
    plt.plot(test_errors, label=f"Test MSE (lr={lr})")

plt.xlabel("Number of Trees")
plt.ylabel("Test MSE")
plt.title("Test MSE vs. Number of Trees")
plt.legend()
plt.show()
```



Interpretation of the Two Most Influential Variables in GBM (g)

In the Gradient Boosting Regressor, the two most influential variables are `AvgSAT` (~27.4% relative influence) and `Pell` (~12.6% relative influence):

- `AvgSAT` **(Selectivity)**: Measures incoming student academic preparation and institution selectivity. Higher average SAT scores strongly predict higher completion rates, as academically well-prepared students are significantly more likely to persist and complete a four-year degree within 150% of normal time.
- `Pell` **(Student Socioeconomic Mix)**: Measures the percentage of undergraduates receiving federal Pell Grants (an indicator of low-income status). Higher Pell shares are strongly associated with lower completion rates, reflecting the greater financial, personal, and academic obstacles faced by economically disadvantaged students.

In [16]:
```
# h
numeric_cols = X_train.select_dtypes(include=['int64','float64']).columns.to

preprocessor = ColumnTransformer(
    transformers=[
        ('num', StandardScaler(), numeric_cols),
        # ('cat', OneHotEncoder(drop='first', handle_unknown='ignore'), cate
    ]
)

lin = Pipeline([
    ('preprocessor', preprocessor),
    ('regressor', LinearRegression())
])

lin.fit(X_train, y_train)
```

```
y_pred_lin = lin.predict(X_test)
test_mse_lin = mean_squared_error(y_test, y_pred_lin)

models['linear'] = {'mse': test_mse_lin}
```

In [17]:
```
display(
    pd.DataFrame.from_dict(models, orient='index')[['mse']]
        .loc[['global_mean', 'control_mean', 'pruned_tree', 'randforest_2',
        .sort_values('mse')
)
```

	mse
randforest_2	153.387885
gbm	160.384251
linear	167.647203
pruned_tree	210.123777
control_mean	397.953675
global_mean	405.409541

Model Comparison and Discussion (h)

Comparing test MSE across all models:

- **Random Forest (`max_features=20`)**: **153.39**
- **GBM (`learning_rate=0.01, n_estimators=1000`)**: **160.38**
- **Linear Model (Standardized & Encoded)**: **167.65**
- **Pruned Decision Tree**: **210.12**
- **Control-Mean Baseline**: **397.95**
- **Global-Mean Baseline**: **405.41**

Observations:

1. **Baselines**: The global and control mean baselines perform poorly (~400 MSE), confirming that institution type alone explains very little of the variance in college graduation rates.
2. **Linear Regression vs. Single Tree**: The linear model (167.65) substantially outperforms the pruned decision tree (210.12). Because completion rate has strong, smooth additive relationships with core predictors (such as test scores, tuition, and demographics), linear regression fits the trend better than a rigid, piecewise-constant tree step function.
3. **Ensemble Methods**: Random Forest (153.39) and GBM (160.38) achieve the best overall accuracy. Ensembles successfully capture nonlinearities and feature interactions while reducing individual tree variance through bagging and boosting.

Part II: Eight Schools

In [18]:
```
# a
results_df = meta_test.copy()
results_df['Control'] = test_df['Control']
results_df['Pell'] = test_df['Pell']
results_df['CompRate'] = y_test
results_df['CompRate_pred'] = y_test_gbm
results_df['residual'] = results_df['CompRate'] - results_df['CompRate_pred'

eight_schools = pd.concat([
    results_df[results_df['Control'] == 'Public'].head(3),
    results_df[results_df['Control'] == 'Private'].head(3),
    results_df[results_df['Control'] == 'Profit'].head(2),
], ignore_index=True)

eight_schools
```

Out[18]:

	ID	Name	Control	Pell	CompRate	CompRate_pred	residual
0	190549	CUNY Brooklyn College	Public	54.5	54.9	47.722002	7.177998
1	234085	Virginia Military Institute	Public	15.3	76.9	73.004059	3.895941
2	209506	Oregon Institute of Technology	Public	21.2	57.2	62.827344	-5.627344
3	168786	Aquinas College	Private	24.0	61.2	63.187769	-1.987769
4	161509	Maine College of Art & Design	Private	32.5	48.3	61.901104	-13.601104
5	183239	Saint Anselm College	Private	11.3	85.3	76.175322	9.124678
6	440749	National American University-Ellsworth	Profit	26.2	16.7	38.070227	-21.370227
7	222938	The Art Institute of Houston	Profit	48.9	40.8	33.704943	7.095057

Final Results

1

I chose the GBM model for this comparison. National American University-Ellsworth has the largest absolute residual of 21.37. The model over-predicted completion for this For-Profit institution (actual: 16.7%, predicted: 38.1%).

2

The errors line up with **neither**. Within every Control sector (Public, Private, and For-Profit) and across both score-reporting and score-missing institutions, the model produces both positive residuals (under-predictions) and negative residuals (over-predictions). Instead, the largest errors appear driven by idiosyncratic, specialized institutions (such as specialized arts colleges and career-focused for-profits) whose unique student bodies are not fully captured by general institutional metrics.

3

Someone might mistakenly conclude that lowering a school's Pell Grant enrollment or increasing spending would causally increase its completion rate, or that schools with high positive residuals are inherently "better quality" institutions, confusing observational association with causal policy impact and ignoring unobserved institutional differences.